Next: Up: Previous:

Any intersecting segments

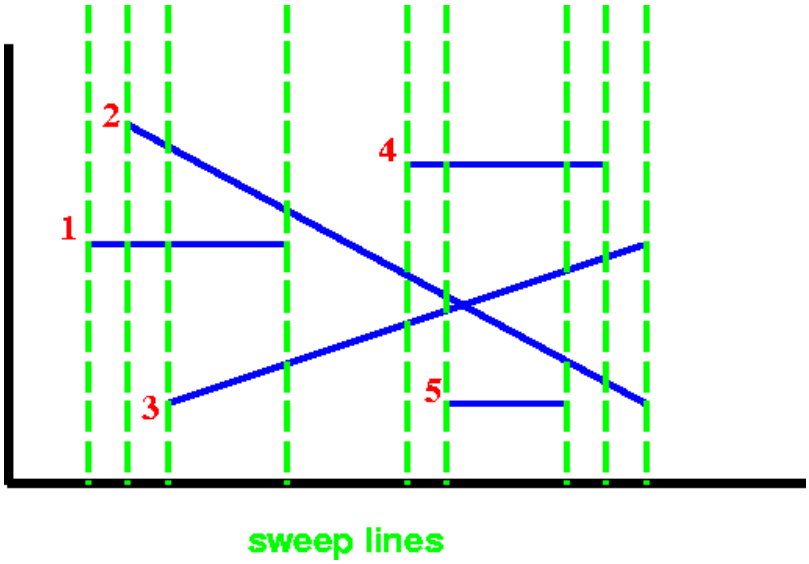
* Sort segments by left endpoints
* Pass a vertical sweep line over the segments
* Put segment in RB tree when hit left endpoint, order by y value
  Check intersection between new and ABOVE and between new and BELOW
* Remove segment from RB tree when hit right endpoint
  Check intersection between ABOVE and BELOW
* If two line segments intersect, they will eventually be neighbors

  Neighbors when one of the segments is added, and the first check will catch the intersection

  Neighbors when another segment is deleted, and the second check will catch the intersection

  The intersecting lines will eventually intersect the sweep line.

  No false positives
* Assume no vertical lines, no 3 lines intersect at same point



Sweep line: RB tree contains 1
Sweep line: RB tree contains 2, 1, no intersect
Sweep line: RB tree contains 2, 1, 3, no intersect
Sweep line: RB tree contains 2, 3, intersect
Sweep line: RB tree contains 2, 4, 3, no intersect
Sweep line: RB tree contains 2, 4, 3, 5, no intersect
Sweep line: RB tree contains 2, 4, 3, no intersect
Sweep line: RB tree contains 2, 3, intersect
Sweep line: RB tree is empty

```
For n segments:
   sort segments                n lg n
   n RB-Inserts                 n lg n
   n RB-Deletes                 n lg n
   O(1) comparison
```

The algorithm takes O(n lg n) time.

[Scanline Algorithm Applet]

Next: Up: Previous: